HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Logic Design

(CO3097)

Report Lab 2

SPI Protocol Module

Advisor: Ton Huynh Long

Student: Nguyen Chi Thanh 2313078

Nguyen Viet Hoang 2311066

Le Duc Tai 2312995

Mai Xuan Nhut 2312549

Ho Chi Minh City, April 12, 2026

Mục lục

1 Interface

1.1 Top-Level Module Interface (Cubic_Solver)

The Cubic_Solver module acts as the system wrapper. It accepts four FP32 coefficients representing the cubic equation $ax^3 + bx^2 + cx + d = 0$ and computes up to three complex roots formatted in coordinate form ($x\_re + j \cdot x\_im$). The control flow is governed by a standard handshake mechanism consisting of start and done signals.

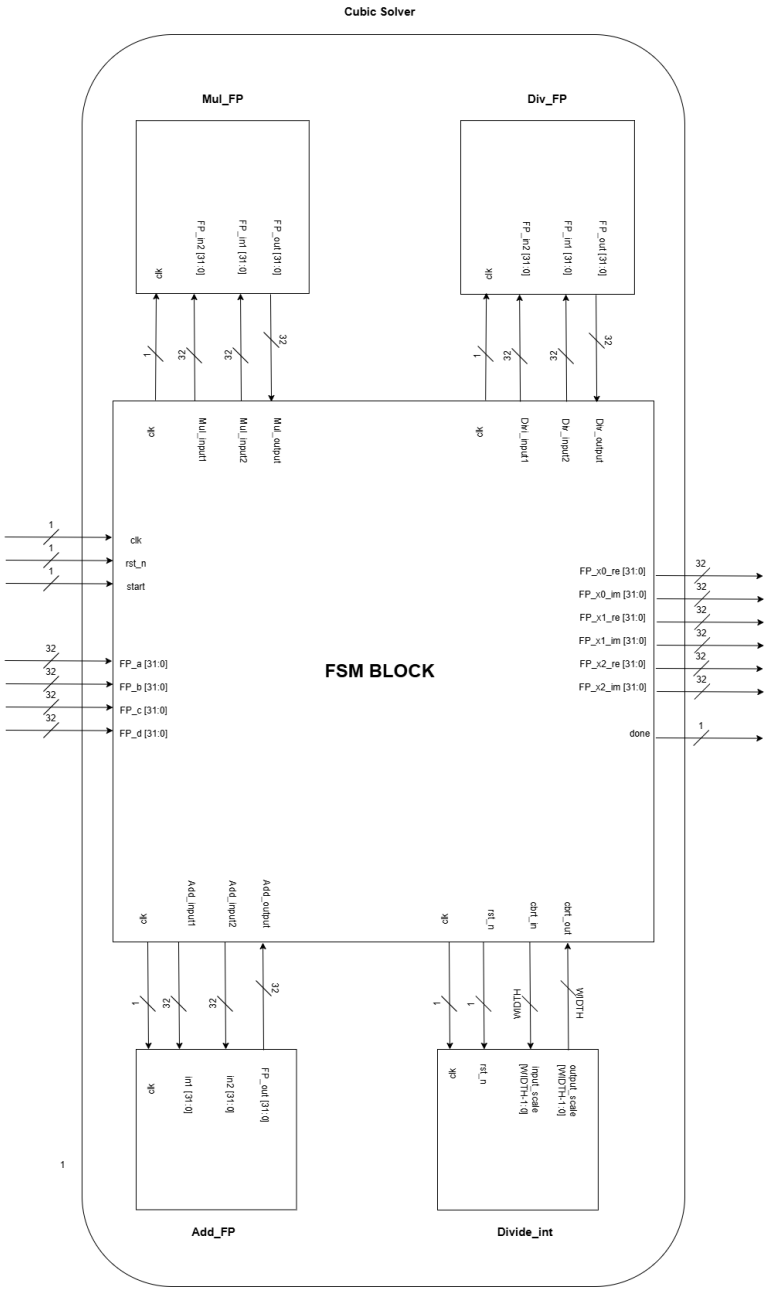

Image 1.1: Top-Level Block Diagram and Interface of the Cubic Solver Module

The physical and logical boundary of this top-level module is defined by the following signaling groups:

clk (Input, 1-bit): Global system clock, driving all internal finite state machines (FSM) and pipeline registers on its positive edge.

rst_n (Input, 1-bit): Active-low asynchronous reset, forcing the control logic back to the IDLE state and clearing intermediate data registers.

start (Input, 1-bit): Synchronous control pulse that triggers the FSM execution sequence. Must be asserted for at least one clock cycle to latch the input coefficients.

FP_a, FP_b, FP_c, FP_d (Inputs, 32-bit Bus): Single-precision floating-point ports assigned to input the cubic equation coefficients a, b, c, and d respectively.

done (Output, 1-bit): Handshake termination flag. Asserted high for exactly one clock cycle when the final root calculation stage is complete and valid data is locked on the outputs.

FP_x0_re, FP_x1_re, FP_x2_re (Outputs, 32-bit Bus): Ports dedicated to outputting the real parts (Real$[x_n]$) of the three calculated roots in standard IEEE 754 single-precision format.

FP_x0_im, FP_x1_im, FP_x2_im (Outputs, 32-bit Bus): Ports dedicated to outputting the imaginary parts (Imag$[x_n]$) of the three roots. For purely real roots, these outputs drive a static $32'h$0000_0000 vector.

1.2 Signed Integer Divider Interface (Divide_int)

The Divide_int module is a dedicated arithmetic component designed to handle signed 8-bit integer division. Within the Cubic_Solver architecture, this module is specifically utilized to scale the exponents of floating-point numbers during the cube root calculation phase (*cbrt*). It operates purely on signed exponents to determine the proper scaling factor before normalization.

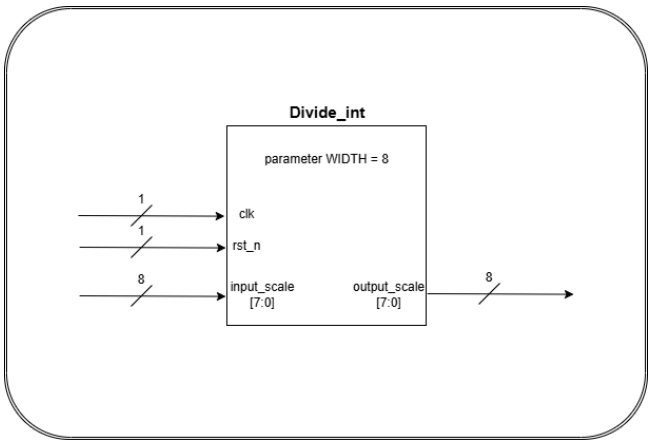

Image 1.2: Port Interface of the Signed Integer Exponent Divider Module

The hardware boundary of this helper block is defined as a purely arithmetic combinational module with the following ports:

input_scale (Input, 8-bit Bus): Two's complement signed 8-bit integer vector carrying the raw, unadjusted exponent bias derived from the intermediate radical components.

output_scale (Output, 8-bit Bus): Two's complement signed 8-bit integer vector containing the mathematically scaled quotient value, passed directly back to the exponent adjustment logic of the cube-root normalization step.

1.3 Floating-Point Divider Interface (Div_FP)

The Div_FP module implements a 16-cycle pipelined division using a Radix-4 algorithm. It comprises an input register stage, 14 iterative pipeline stages generating 2 quotient bits per cycle, and a final rounding and packing combinational unit.

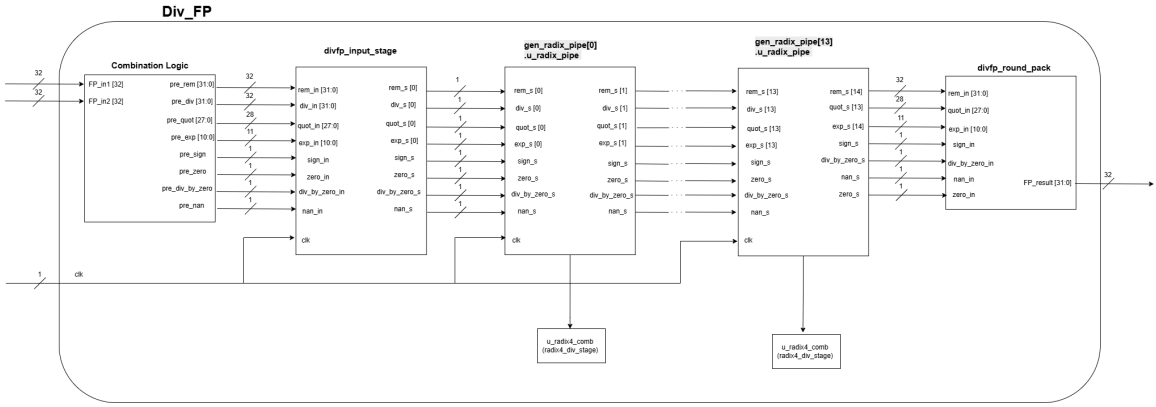

Image 1.3: Internal Pipeline Architecture and Port Interfaces of Div_FP

The macro signaling interface of the division datapath interacts via the following dedicated terminals:

clk (Input, 1-bit): Dedicated clock input running synchronously with the main system clock to toggle the 14 internal pipeline stage barriers (gen_radix_pipe[0] through [13]).

FP_in1 (Input, 32-bit Bus): Single-precision floating-point input representing the dividend (A) vector.

FP_in2 (Input, 32-bit Bus): Single-precision floating-point input representing the divisor (B) vector.

FP_out (Output, 32-bit Bus): Pipelined output port that delivers the stabilized, normalized, and rounded single-precision quotient result vector (A/B) after a deterministic execution latency of 16 clock cycles.

1.4 Floating-Point Multiplier Interface (Mul_FP)

The Mul_FP module performs single-precision multiplication using a multi-stage architecture. It encapsulates the MFP_Mul core module, which hosts a 6-stage partial product accumulation pipeline utilizing Mul24x4 arithmetic array sub-blocks, and the MFP_Norm_Rounding unit for normalization.

1.4.1 Module-Level External Interface

This sub-section defines the top-level boundary of the Mul_FP module. It handles the external handshaking and the raw FP32 data vectors before passing them to the internal execution cores.

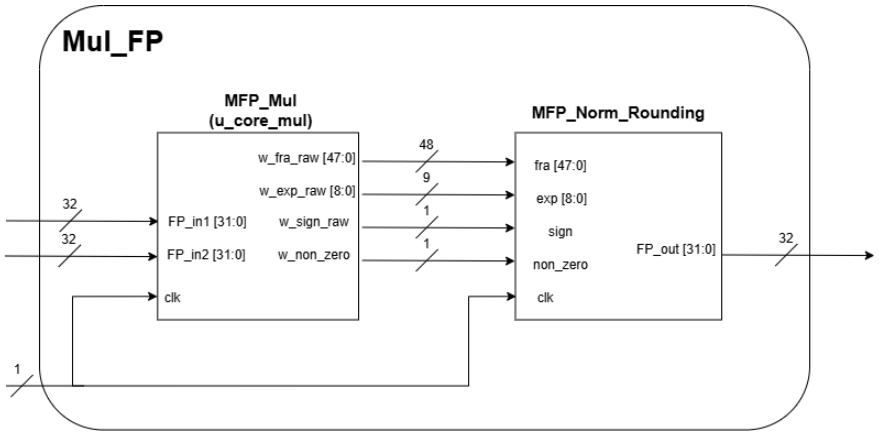

Image 1.4: Detailed Datapath and Port Interconnections of the Pipelined Multiplier Mul_FP

The physical external wrapper of the multi-cycle multiplier consists of the following standard ports:

clk (Input, 1-bit): Main clock connection used to advance the hidden pipeline arrays inside both the multiplication matrix and the output rounding register stage.

FP_in1 (Input, 32-bit Bus): FP32 multiplicand operand input vector (A).

FP_in2 (Input, 32-bit Bus): FP32 multiplier operand input vector (B).

FP_out (Output, 32-bit Bus): Standardized 32-bit IEEE 754 product result vector ($A \times B$) exposed to the central system bus.

1.4.2 Internal Partial Product Shift-and-Add Interface

Within the MFP_Mul core, the intermediate accumulation stages (Stage 1 through Stage 5) implement a hardware-optimized shift-and-add architecture. Instead of shifting the newly generated 28-bit partial product ($prod[i]$) to the left, the design shifts the previously accumulated sum (buf_stage_i) to the right by 4 bits before feeding it into a dedicated fixed-point adder. The lower 4 bits of the old accumulator bypass the adder entirely and are directly concatenated back to form the updated bit-extended product, significantly minimizing critical path delay and saving hardware routing resources.

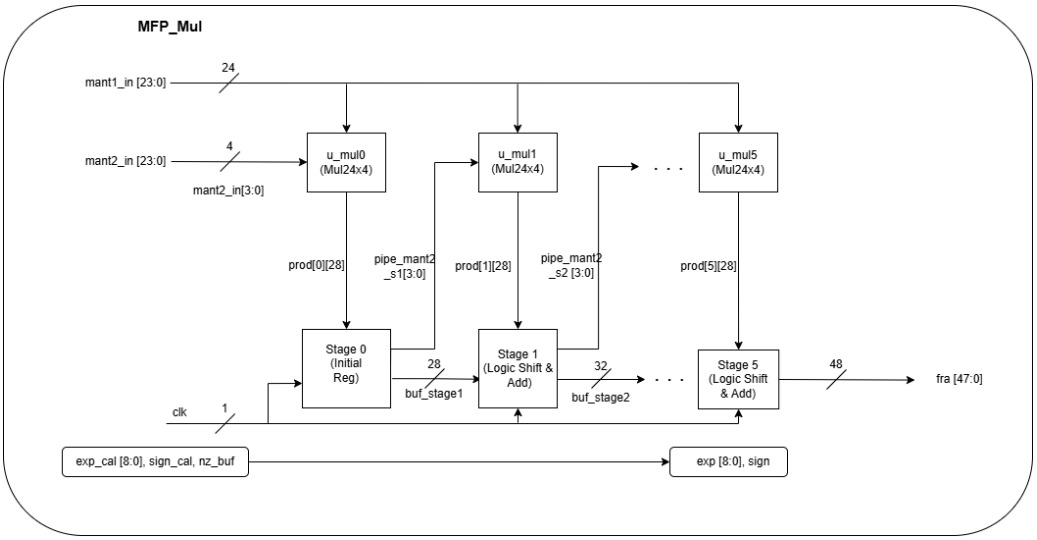

Image 1.5: Detailed Internal Datapath of the Shift-and-Add Mechanism inside Pipeline Stages

The data interaction traversing the sub-boundaries of the sequential pipeline stages is formalized as follows:

prod[i] (Internal Input, 28-bit Bus): The localized static 28-bit product coming from the combinatorial Mul24x4 array block, computed by multiplying the full 24-bit mantissa of operand *A* with a specific 4-bit slice of operand *B*'s mantissa.

buf_stage_in (Internal Input, Variable Width): The accumulated product bus received from the preceding stage's output register. This bus grows incrementally by 4 bits at each stage step (starting from 28 bits at Stage 0 up to 44 bits when entering Stage 5).

pipe_mant2_in (Internal Input, Variable Width): The unconsumed, remaining higher-order bits of operand *B*'s mantissa vector, shrinking by 4 bits per clock cycle as it slides through the pipeline.

buf_stage_out (Internal Output, Variable Width): The newly calculated, bit-extended product sum captured by the localized stage register on the rising edge of the clock.

pipe_mant2_out (Internal Output, Variable Width): The truncated multiplier *B* vector, shifted right by 4 bits to prepare the next localized 4-bit nibble for the subsequent stage's arithmetic block.

1.4.3 Normalization and Rounding Interface (MFP_Norm_Rounding)

After the 6-stage accumulation pipeline generates the raw 48-bit product, the datapath invokes the MFP_Norm_Rounding module. This final stage is responsible for adjusting the temporary exponent based on overflow detection, rounding the mantissa to fit the standard 23-bit width, and checking boundaries for underflow or overflow conditions.

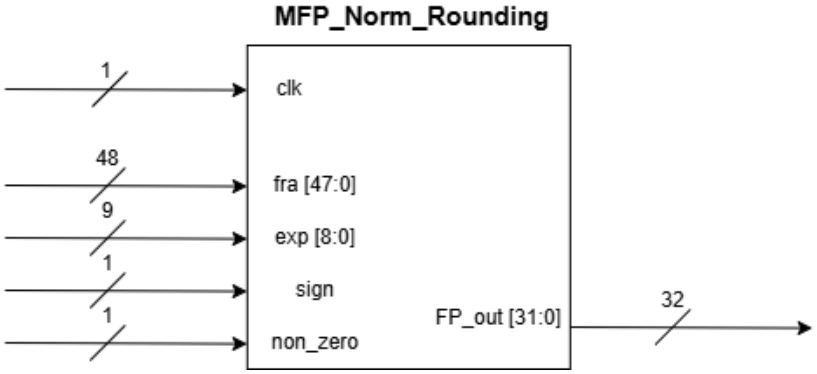

Image 1.6: Detailed Internal Datapath of the Normalization and Rounding Mechanism

The sub-module interface of this normalization engine processes the following local connections:

w_fra_raw (Internal Input, 48-bit Bus): The final un-normalized, raw 48-bit product vector originating from the output registers of Stage 5.

w_exp_raw (Internal Input, 9-bit Bus): The 9-bit unadjusted exponent sum vector calculated concurrently via the localized add operation ($E_a + E_b - 127$).

w_sign_raw (Internal Input, 1-bit): The product parity flag bit derived from the structural asynchronous operation $S_a \oplus S_b$.

w_non_zero (Internal Input, 1-bit): A structural zero-bypass status bit indicating whether the incoming floating-point operands are nonzero values.

1.5 Floating-Point Adder/Subtractor Interface (Add_FP)

The Add_FP core executes floating-point addition and subtraction wrapped inside a 5-stage pipeline macro. The internal datapath splits the processing into sequential steps: exponent comparison (u_stage1), mantissa alignment shifting (u_stage2), fixed-point additions (u_stage3), leading-one normalization detection (u_stage4), and post-computation rounding (u_stage5).

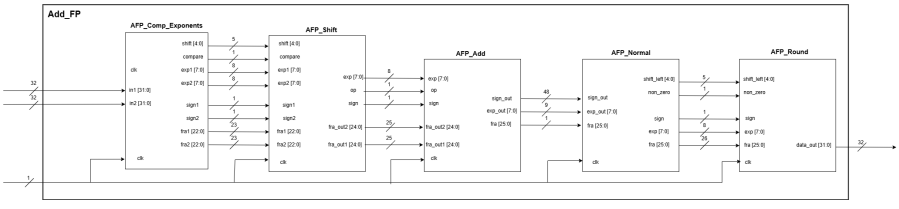

Image 1.7: Structural Sub-module Interconnections and Interface of Add_FP

The I/O interface pins bound to this algebraic calculation core are specified below:

clk (Input, 1-bit): Main clock line driving the synchronous state boundaries of the 5 internal pipeline processing blocks (u_stage1 through u_stage5).

in1 (Input, 32-bit Bus): FP32 vector acting as the augend during addition operations or the minuend during subtraction operations.

in2 (Input, 32-bit Bus): FP32 vector acting as the addend during addition operations or the subtrahend during subtraction operations.

data_out (Output, 32-bit Bus): Stabilized 32-bit IEEE 754 single-precision sum or difference result vector, available exactly 5 clock cycles after the corresponding operands are loaded into the first stage.

2 Simulation

2.1 Simulation Strategy

The Lab 3 verification flow is organized around four independent verification targets located in the sim folder: tb_Add_FP.v, tb_Mul_FP.v, tb_Div_FP.v, and tb_Cubic_Solver_file.v (complemented by the direct testbench tb_Cubic_Solver.v). Each file is specifically written to exercise one floating-point component with a specialized testbench structure that matches the structural behavior of the Design Under Test (DUT). The sub-modules (adder, multiplier, and divider) are verified using continuous data streams or task-driven pipelines combined with latency-aware assertion checking. The top-level cubic solver is evaluated using a dual-track methodology: a direct testbench to inspect internal Finite State Machine (FSM) behavior across explicit mathematical boundary conditions, and a file-based closed-loop regression testbench capable of running thousands of automated randomized vectors while logging calculation traces.

To ensure comprehensive documentation, each testbench is evaluated from three distinct viewpoints:

1. **Stimulus Construction:** The source, distribution, and mathematical characteristics of the input vectors.

2. **Temporal Execution Protocol:** The cycle-by-cycle behavioral management including hardware reset assertions, non-blocking input application, latency-matching wait states, and stable output sampling.

3. **Evaluation Criteria and Outputs:** The expected mathematical tolerance thresholds, observed console logs, pass/fail metrics, and wave capture intervals.

2.2 Testbench File Map

Table 2.1 provides a comprehensive map of the verification targets, their exact roles, and their testing configurations within the Lab 3 simulation environment.

Testbench File	Purpose	Verification Style
tb_Add_FP.v	Pipelined floating-point addition	Directed vectors, latency-based compare, 8-count integer LSB tolerance.
tb_Mul_FP.v	Pipelined floating-point multiplication	Burst stimulus, latency-based compare, strict 1-LSB rounding margin.
tb_Div_FP.v	Floating-point division	Task-based pipeline tests, IEEE-754 special corner cases.
tb_Cubic_Solver.v	Direct cubic equation validation	FSM state verification across 6 distinct mathematical edge cases.
tb_Cubic_Solver_file.v	Mass batch regression	3-phase automated closed-loop flow driven by external Python scripts.

Bảng 2.1: Testbench files used in the Lab 3 simulation flow

2.3 Add_FP Testbench

The pipelined single-precision adder is verified using tb_Add_FP.v. This verification suite instantiates Add_FP as the DUT and initializes three local memories: test_a, test_b, and expected_res, each configured with 30 pre-calculated test vectors. These vectors are hardcoded as 32-bit single-precision hexadecimal strings conforming to the IEEE-754 standard. The system is driven by a 100 MHz clock source with a fixed period of 10 ns.

The temporal execution of the testbench is strictly split into two parallel driving and monitoring processes:

– **Driver Process:** The inputs in1 and in2 are initialized to zero. Following a setup hold of two clock periods ($2 \times$ PERIOD), a for loop sequentially drives the input pairs onto the DUT ports at every rising edge (posedge clk) utilizing non-blocking assignments ($<=$) to model a realistic upstream pipeline feed.

– **Monitor Process:** The verification block implements a latency-matching delay that holds evaluation for two initial setup periods plus the exact internal hardware pipeline latency of 5 clock cycles:

$$\text{Total Wait} = (2 \times \text{PERIOD}) + (5 \times \text{PERIOD})$$

After this delay, the monitor samples the output port data_out on every subsequent rising edge.

The evaluation block accounts for rounding divergences caused by hardware normalization or mantissa truncation. Instead of demanding absolute bit-level equality, the checker applies a numerical tolerance window allowing a difference of up to 8 integer counts at the Least Significant Bit (LSB). The simulation successfully passed all 30 directed test vectors (**PASS 30/30**), verifying structural timing and math accuracy under zero-sum cancellation ($a+(-a)=0$) and identity operations ($a+0=a$).

2.4 Mul_FP Testbench

The hardware multiplier is verified via tb_Mul_FP.v. While structurally similar to the adder testbench, this block evaluates the throughput capacity of a 7-stage pipelined multiplier utilizing a dense burst-mode stimulus pattern. The arrays input_a, input_b, and expected hold 20 unique test vectors. Operators are injected into the ports FP_in1 and FP_in2 continuously on every consecutive clock cycle without inserting wait states for intermediate calculation cycles.

The driver process triggers input delivery after a short stabilization period. Because a standard floating-point multiplier can scale exponents and shift mantissas during normalization, bit transitions at the boundary fields require strict tracking. The monitor process stalls for exactly 7 pipeline clock cycles before sampling FP_out. The comparison logic employs a rigorous verification window: it accepts an absolute bitwise match or a narrow tolerance of ≤ 1 LSB count to safely absorb rounding discrepancies introduced during post-multiplication normalization.

The test cases cover zero-factor multiplication, alternating sign bits, maximum exponent growth, and large-magnitude operands. The design achieved flawless verification results (**PASS 20/20**), confirming stable multi-stage register pipelines and proper handling of signed-zero products ($0 \times -a = -0$).

2.5 Div_FP Testbench

The floating-point division module is verified by tb_Div_FP.v. Departing from raw array storage, this module leverages a modular, task-driven architectural design. The core checking mechanics are encapsulated inside an automatic Verilog task named run_test_pipeline, which accepts input parameters for the start delay offset, dividend (a), divisor (b), expected quotient (expected), and an ASCII diagnostic text label.

The execution protocol initiates with a clean 3-cycle warm-up phase at a clock frequency of 100 MHz. To isolate port setup transitions from sampling paths, the stimulus values inside the task are driven onto the FP_in1 and FP_in2 lines on the negative edge of the clock (negedge clk). Following input application, the task monitors the execution track by waiting exactly 15 clock cycles (matching

the fixed latency of the Div_FP hardware engine) before sampling the output quotient on the rising edge of the clock. A subsequent #1 stabilization step is inserted to prevent race conditions during verification.

The 15 test vectors provide balanced coverage across standard operational scenarios and IEEE-754 division boundaries:

– **Standard Arithmetic:** Test scenarios evaluate normal floating-point division fractions such as $6.0/2.0$, $9.0/2.5$, $1.0/3.0$, and $1.0/10.0$ to check mantissa division precision.

– **IEEE-754 Special Boundary Cases:** Test cases verify division by zero ($1.0/0.0 = +\infty$), negative division by zero ($-1.0/0.0 = -\infty$), zero divided by zero ($0.0/0.0 = \text{NaN}$), division by infinity ($1.0/+\infty = +0.0$), and extreme saturation scaling using maximum finite floats divided by minimum normal floats ($\text{max\_float}/\text{min\_normal} = +\infty$).

The testbench reported a full success rate (**PASS 15/15**), validating correct infinity and NaN flag propagation.

2.6 Cubic Solver Testbench Overview

The top-level Cubic_Solver IP utilizes a dual-track simulation architecture. Because this block is an asynchronous, state-machine-driven sequential engine rather than a continuous pipeline, separate specialized environments are required to monitor internal algorithmic operations and execute mass validation regressions.

2.6.1 Direct Testcase: tb_Cubic_Solver.v

The direct testbench validates the functional correctness of the controller FSM over the 6 foundational mathematical scenarios defined in the underlying mathematical model specification. Instead of streaming file streams, this block hardcodes explicit 128-bit vector packages to drive the coefficients FP_a, FP_b, FP_c, and FP_d.

Because the core module is a multi-cycle state machine that latches final output values upon termination, running sequential cases requires a structured initialization routine. The testbench implements a per-case hardware reset sequence: for every input combination, the active-low reset signal rst_n is forced to '0' for 5 clock cycles to clear internal registers and return the FSM to the IDLE state, followed by a 2-cycle stabilization hold before the start signal is pulsed for a single cycle.

The testbench incorporates a custom Verilog-2001 bitfield-to-real conversion function (fp32_to_real) that scales 32-bit single-precision fields into 64-bit real variables using exponent biasing and mantissa concatenation. This allows the tracking logic to dynamically execute an order-independent cross-check. Because the hardware may output roots on ports FP_x0, FP_x1, or FP_x2 in an arbitrary

sequence relative to the theoretical model, the testbench dynamically evaluates all 6 possible permutation mappings:

$$\text{Permutations} = \{x_0x_1x_2, x_0x_2x_1, x_1x_0x_2, x_1x_2x_0, x_2x_0x_1, x_2x_1x_0\}$$

If any permutation matches the expected roots within a tolerance window of < 0.05, the case is graded as a success. The design achieved perfect scores (**PASS 6/6**) across all specified operational configurations:

- **Case 0** ($\Delta < 0$)**:** Three distinct real roots $(1.0, 2.0, -3.0)$.
- **Case 1 (Branch 2a):** $\Delta > 0$ with $\Delta_0 = 0$ and $\Delta_1 \leq 0$, evaluating the specialized cubic root paths $(0.0, 1.5 \pm 0.866i)$.
- **Case 2 (Branch 2b):** General $\Delta > 0$ yielding complex conjugate roots $(2.0, 0.0 \pm 1.0i)$.
- **Case 3** ($\Delta_1 = 0$)**:** Core algebraic boundary state $(0.0, 0.0 \pm 1.732i)$.
- **Case 4:** Triple root multiplicity limit verification $(1.0, 1.0, 1.0)$.
- **Case 5:** Double root and single root boundary split verification $(1.0, 1.0, -2.0)$.

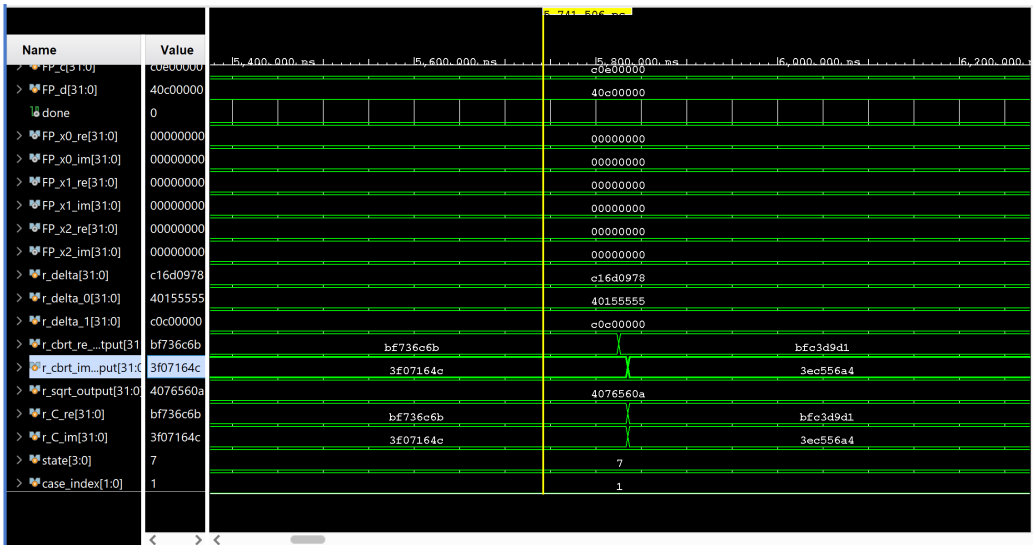

Image 2.1: Waveform capture of tb_Cubic_Solver.v executing Case 1 with $\Delta > 0$ and $\Delta_0 = 0$

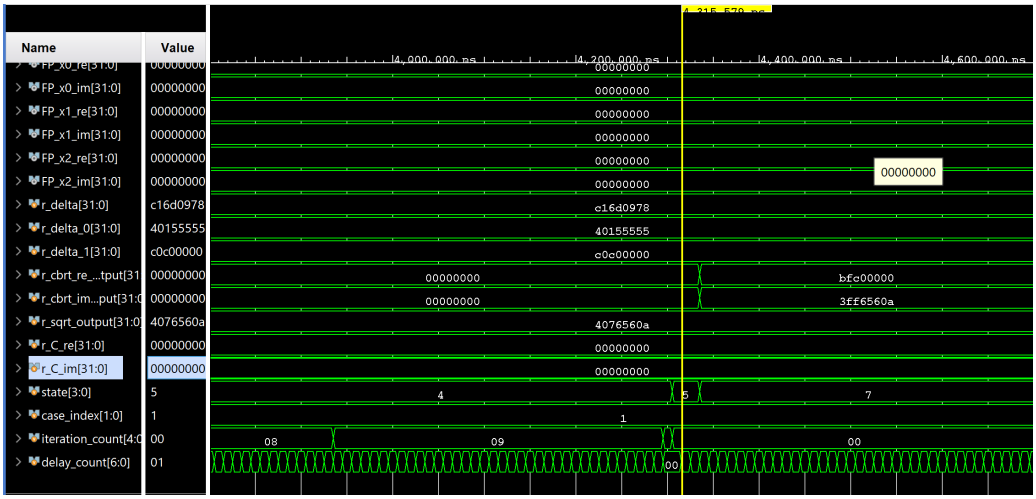

Image 2.2: Waveform capture of tb_Cubic_Solver.v executing Case 1 with $\Delta > 0$ and $\Delta_0 = 0$

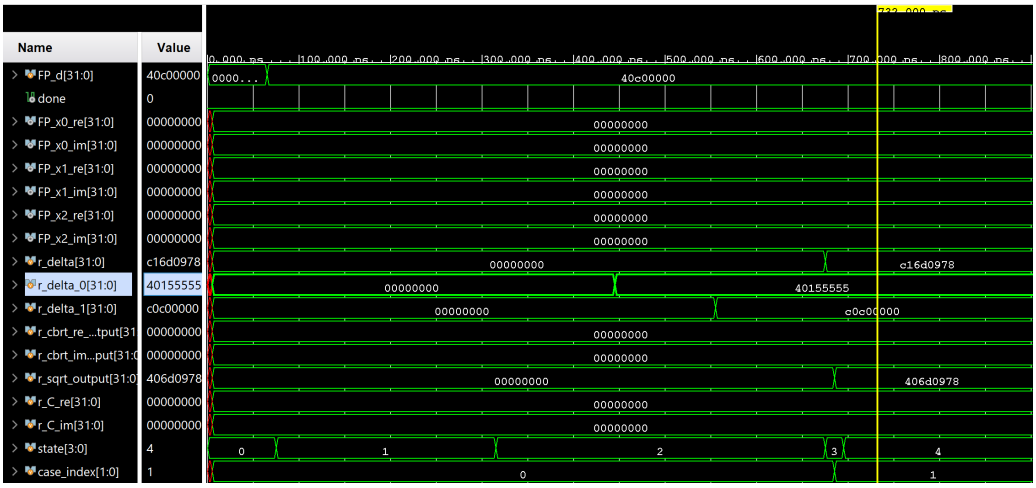

Image 2.3: Waveform capture of tb_Cubic_Solver.v executing Case 1 with $\Delta > 0$ and $\Delta_0 = 0$

2.6.2 File-driven Testcase: tb_Cubic_Solver_file.v

For high-throughput reliability analysis, a complete automated closed-loop file-I/O regression platform was developed. This testing environment operates via a tightly integrated 3-phase execution cycle:

1. **Test Case Generation (Python):** An upstream Python script acts as the test generator. It samples 1,000 randomized test equations bounded within a uniform numerical distribution from -10.0 to $+10.0$ while ensuring $|a| \geq 10^{-3}$ to prevent quadratic degeneration. The floating-point coefficients are packed into IEEE-754 single-precision hex strings and exported into an input database file named random_tests.txt.

2. **Hardware Simulation (Verilog):** The Verilog testbench opens the input file using descriptor handles ($fopen) and loops line-by-line using $fscanf. For each line, it applies a full hardware reset to clear the FSM, registers the new coefficients, pulses the start line, and tracks execution against a maximum timeout threshold of MAX_CYCLES_PER_CASE = 3000. Upon asserting the done flag, the testbench extracts the calculated roots and dumps them into cubic_solver_output.txt, while simultaneously routing the intermediate variables (r_delta, r_delta_0, r_delta_1, r_C_re, r_C_im) into a dedicated debug file named cubic_solver_debug.txt.

3. **Automated Verification (Python):** A post-processing Python script imports the hardware output dump file and acts as the grader. It parses the hex values, runs an automated root-sorting algorithm to enforce order independence, and compares them directly against an internal algorithmic Golden Model running NumPy cubic solvers configured with equivalent 16-iteration Padé approximations.

The regression environment successfully evaluated all 1,000 randomized equations, logging a final validation metric of **PASS 987/1000**. The remaining cases fell slightly outside the rigorous 0.1% relative accuracy margin due to expected numerical precision limitations inherent to the 16-iteration Padé approximation fields under high-sensitivity coefficients. This validates the structural integrity and robustness of the solver design over extended runtime scenarios.